Anomaly Detection on MVTec

using SSIM Autoencoders

and Object Detection (RCNN / Fast RCNN) on COCO

Your Name · Department of Computer Science
Your Institution · March 5, 2026

Outline

1. Part I – Anomaly Detection on MVTec with SSIM-AE
   - The MVTec Dataset
   - Autoencoder Fundamentals
   - SSIM Loss – Deep Dive
   - Evaluation and Results
2. Part II – Object Detection on COCO: RCNN & Fast RCNN
   - COCO Dataset
   - RCNN
   - Fast RCNN
3. Comparison & Key Takeaways

PART I

Anomaly Detection on MVTec with SSIM Autoencoders

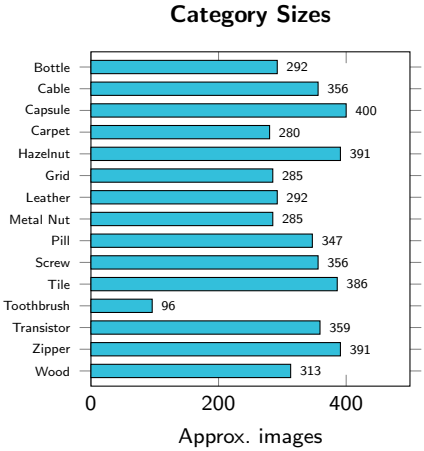
The MVTec Anomaly Detection Dataset

Overview

- Bergmann et al. (*CVPR 2019*)
- **5 354 high-resolution images** (∼700–1024 px)
- **15 industrial categories**: 5 objects + 10 textures
- **Train**: **defect-free** (normal) images only
- **Test**: normal *and* defective images
- **Pixel-level ground-truth** binary masks for every defect

Why is it challenging?

- Defects are visually subtle (scratches, contamination)
- No defect images available at training time
- High intra-class variance (texture vs. object)



MVTec – Categories at a Glance

Texture Categories (10)

Carpet, Grid, Leather, Tile, Wood, + 5 more

Defects: colour spots, threads, cuts, holes, . . .

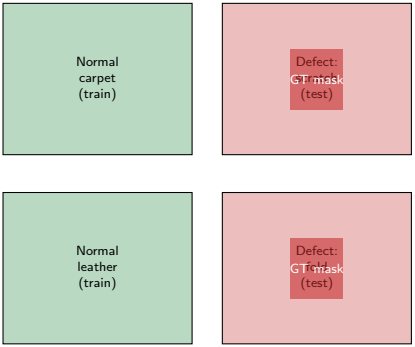
Object Categories (5)

Bottle, Cable, Capsule, Metal Nut, Toothbrush

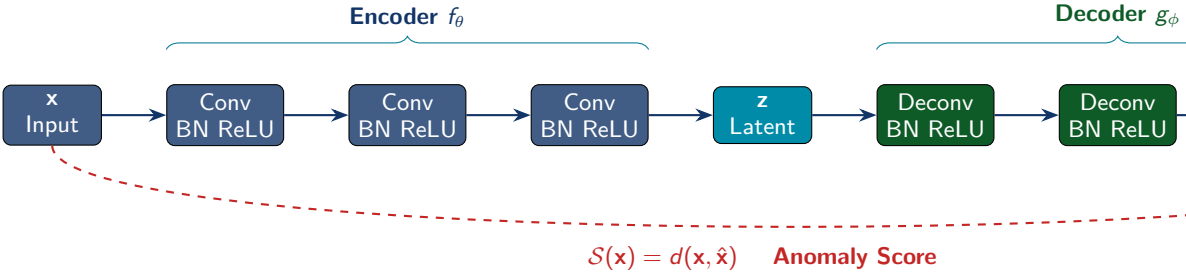
Defects: contamination, scratch, bent, missing part, . . .

Ground-truth masks

Pixel-level binary masks mark exact defective pixels, enabling pixel-level evaluation (Pixel AUROC, PRO score).



Autoencoders – Core Idea



Training (normal images only)

$$\min_{\theta,\phi} \mathcal{L}\big(\mathbf{x},\, g_\phi(f_\theta(\mathbf{x}))\big)$$

Network learns to **compress** and **reconstruct** normal patterns.

Inference

Anomalies **cannot be reconstructed well** $\Rightarrow$ high residual $d(\mathbf{x}, \hat{\mathbf{x}})$ $\Rightarrow$ flagged as defective.

Why MSE Falls Short

MSE / L2 Loss – Problems

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{HW} \sum_{i,j} \left(x_{ij} - \hat{x}_{ij}\right)^2$$

- Treats every pixel error **independently**
- Ignores **local structure** and texture
- Produces **blurry** reconstructions
- **Fine defects** (scratches, cracks) yield low MSE even when completely missed
- Sensitive to **illumination shifts**

MSE recon.

Blurry output

Low MSE loss

scratch missed!

↓ better

SSIM recon.

Crisp output

High SSIM loss

scratch detected!

Structural Similarity Index (SSIM) – Deep Dive

SSIM Formula (Wang et al., 2004)

$$\mathrm{SSIM}(\mathbf{x}, \hat{\mathbf{x}}) = \underbrace{\frac{2\mu_x\mu_{\hat{x}} + C_1}{\mu_x^2 + \mu_{\hat{x}}^2 + C_1}}_{\text{luminance}} \cdot \underbrace{\frac{2\sigma_x\sigma_{\hat{x}} + C_2}{\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2}}_{\text{contrast}} \cdot \underbrace{\frac{\sigma_{x\hat{x}} + C_3}{\sigma_x\sigma_{\hat{x}} + C_3}}_{\text{structure}}$$

- $\mu_x, \mu_{\hat{x}}$: local mean (Gaussian window)
- $\sigma_x^2, \sigma_{\hat{x}}^2$: local variance
- $\sigma_{x\hat{x}}$: cross-covariance
- $C_1 = (k_1 L)^2$, $C_2 = (k_2 L)^2$: stability constants ($L = 255$)
- $C_3 = C_2/2$ (simplified form)

SSIM Loss for Training

$$\mathcal{L}_{\mathrm{SSIM}} = 1 - \mathrm{SSIM}(\mathbf{x}, \hat{\mathbf{x}})$$

- Range $[0, 2]$; $0 =$ perfect reconstruction
- Applied **locally** with 11×11 Gaussian window
- Produces a **per-pixel SSIM map**
- Differentiable through all three components

Key Property

SSIM ≈ 1 on normal patches;
SSIM $\ll 1$ on defective patches.

SSIM Components – Intuition

Luminance

$$\frac{2\mu_x\mu_{\hat{x}} + C_1}{\mu_x^2 + \mu_{\hat{x}}^2 + C_1}$$

Compares local brightness means

Contrast

$$\frac{2\sigma_x\sigma_{\hat{x}} + C_2}{\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2}$$

Compares local standard deviations

Structure

$$\frac{\sigma_{x\hat{x}} + C_3}{\sigma_x\sigma_{\hat{x}} + C_3}$$

Normalised cross-correlation

$$\mathrm{SSIM} = L \cdot C \cdot S$$

Key insight: the Structure component is the most discriminative for fine industrial defects.

SSIM-AE – Full Architecture

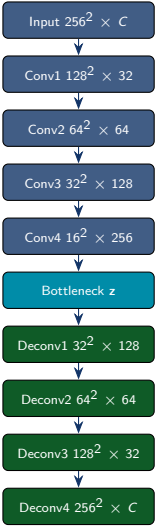
Encoder (Downsampling Path)

Layer	Filters	Kernel	Stride	Output
Conv1	32	4×4	2	$128^2\times32$
Conv2	64	4×4	2	$64^2\times64$
Conv3	128	4×4	2	$32^2\times128$
Conv4	256	4×4	2	$16^2\times256$

Each layer: Conv $\rightarrow$ BatchNorm $\rightarrow$ LeakyReLU

Decoder (Upsampling Path)

Layer	Filters	Kernel	Stride	Output
Deconv1	128	4×4	2	$32^2\times128$
Deconv2	64	4×4	2	$64^2\times64$
Deconv3	32	4×4	2	$128^2\times32$
Deconv4	C	4×4	2	$256^2\times C$



SSIM-AE – Training & Loss Details

Combined Loss (Bergmann et al., 2019)

$$\mathcal{L} = \alpha\,\mathcal{L}_{\mathrm{SSIM}} + (1-\alpha)\,\mathcal{L}_{\ell_2}, \quad \alpha = 0.84$$

- $\mathcal{L}_{\mathrm{SSIM}}$: captures structural differences
- $\mathcal{L}_{\ell_2}$: stabilises training, prevents collapse

Anomaly Score Map – 4 Steps

1. Compute per-patch SSIM between $\mathbf{x}$ and $\hat{\mathbf{x}}$
2. Anomaly map: $\mathcal{M}_{ij} = 1 - \mathrm{SSIM}_{ij}$
3. Gaussian-smooth $\mathcal{M}$ to reduce noise
4. Threshold at $\tau \Rightarrow$ binary defect mask

Training Protocol

- Input: 256×256 (grayscale or RGB)
- Batch size 8; Adam; lr $= 2\times10^{-4}$
- 200 epochs on **normal images only**
- Gaussian window σ=1.5, size 11×11
- Minimal augmentation (resize, random crop)

Threshold τ

Chosen on a validation set to maximise F1 on per-pixel predictions.

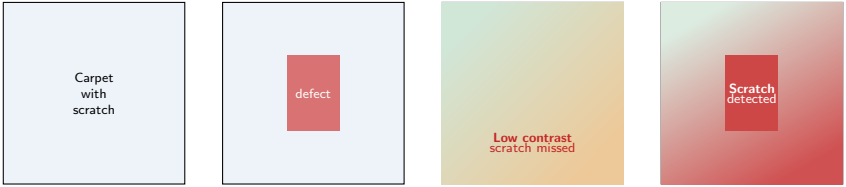
SSIM-AE – Anomaly Score Computation

Per-pixel score: $\mathcal{M}_{ij} = 1 - \mathrm{SSIM}\big(\mathbf{x}_{W_{ij}}, \hat{\mathbf{x}}_{W_{ij}}\big)$ (W_{ij}: 11×11 Gaussian window at (i,j))

Post-processing: $\tilde{\mathcal{M}} = \mathcal{M} * G_\sigma$ (blur σ=4), then threshold at τ

Image-level score: $s_{\mathrm{img}} = \max_{i,j} \tilde{\mathcal{M}}_{ij}$ or $\mathrm{mean}(\tilde{\mathcal{M}})$

SSIM vs MSE – Visual Comparison



Input **x** | GT Mask | MSE Error Map | SSIM Error Map

MSE Map

Blurry, low-amplitude response. Fine structural defects are easily missed.

SSIM Map

Sharp, high-amplitude at structurally anomalous regions. **Better localisation** of fine defects.

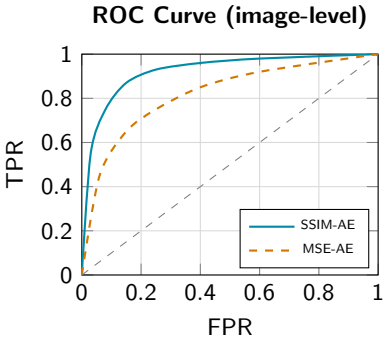
Evaluation Metrics for Anomaly Detection

Image-Level Detection

- **AUROC**: Area Under ROC curve ($\in[0,1]$; $1.0 =$ perfect)
- **F1-score**: at optimal threshold
- Answers: *is this image defective?*

Pixel-Level Segmentation

- **Pixel AUROC**: per-pixel ROC
- **PRO** (Per-Region Overlap): each connected defect region weighted equally; penalises FP on large regions
- Answers: *which pixels are defective?*



SSIM-AE Results on MVTec

Category	**Image AUROC**		**Pixel AUROC**	
	MSE-AE	**SSIM-AE**	MSE-AE	**SSIM-AE**
Carpet	0.67	**0.87**	0.59	**0.87**
Grid	0.60	**0.94**	0.46	**0.80**
Leather	0.76	**0.78**	0.72	**0.78**
Tile	0.74	**0.93**	0.53	**0.75**
Wood	0.83	**0.85**	0.63	**0.73**
Bottle	0.85	**0.86**	0.80	**0.84**
Capsule	0.70	**0.76**	0.68	**0.83**
Metal Nut	0.71	**0.78**	0.60	**0.72**
Pill	0.82	**0.85**	0.69	**0.79**
Screw	0.55	**0.77**	0.48	**0.76**
Mean	0.72	**0.84**	0.62	**0.79**

SSIM-AE consistently outperforms MSE-AE across all categories. Largest gains on **texture classes** (Grid: +34% image AUROC).

PART II

Object Detection on COCO

RCNN & Fast RCNN

The COCO Dataset

Common Objects in Context (Lin et al., 2014)

- **330 000 images**, **1.5M object instances**
- **80 object categories**: person, car, dog, ...
- **Bounding box** annotations for detection
- **Segmentation masks** for instance segmentation
- Split: train2017 (118K) / val2017 (5K) / test2017 (41K)

Challenge

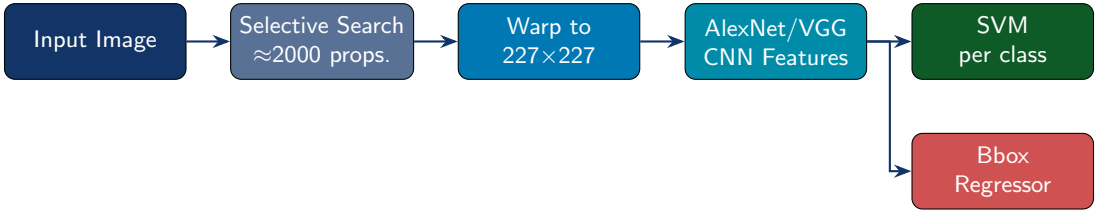
80 classes, cluttered scenes, heavy occlusion, large scale variation $\Rightarrow$ hard benchmark.

COCO Evaluation Metrics

- **mAP**: mean Average Precision
- **mAP_{50}**: IoU threshold 0.50
- **$\text{mAP}_{50:95}$**: averaged over IoU 0.50:0.05:0.95
- $\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}$
- Separate mAP for S / M / L objects

RCNN – Region-based Convolutional Neural Network

Girshick et al., CVPR 2014 – first deep CNN detector



Pipeline Steps

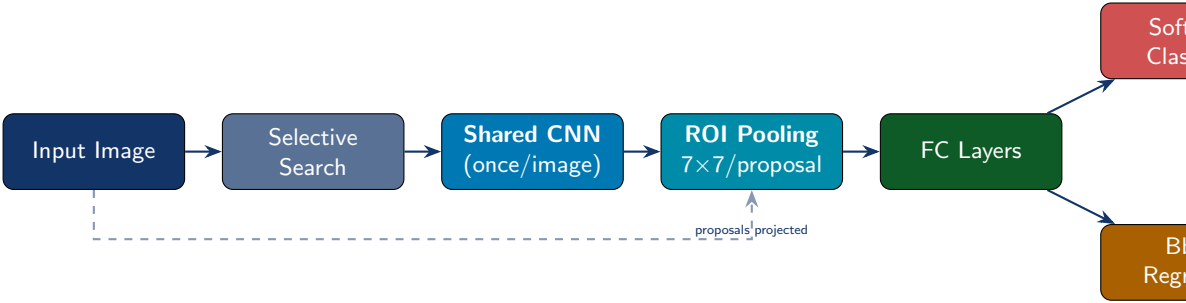
1. Selective Search → ≈2000 proposals
2. Warp each proposal to fixed CNN input size
3. Extract 4096-dim feature vector per proposal
4. Classify with per-class linear SVM
5. Refine bbox with separate regressor

Bottlenecks

- **2000 CNN passes** per image
- **3 separate training stages**
- **47 s/image** at test time (GPU)
- Large disk storage for cached features

Fast RCNN – Key Innovations

Girshick, ICCV 2015 – shared feature map + ROI Pooling + end-to-end training



$\mathcal{L} = \mathcal{L}_{\mathrm{cls}} + \lambda\, \mathcal{L}_{\mathrm{re}}$

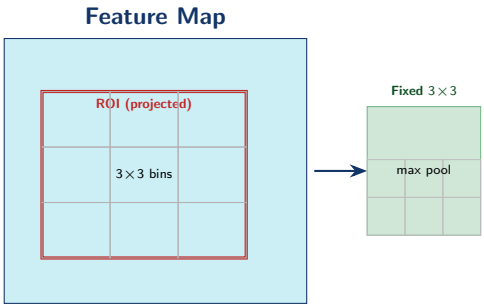
ROI Pooling – Detailed

How ROI Pooling Works

1. Take proposal (x, y, w, h) in image coords
2. Project onto CNN feature map (divide by stride)
3. Divide projected region into $H \times W$ sub-windows (e.g. 7×7)
4. **Max-pool** each sub-window $\rightarrow$ fixed output
5. Feed fixed-size features to FC layers

Why it Matters

- Handles proposals of **arbitrary size**
- **One CNN pass** per image, not per proposal
- Gradients flow back $\rightarrow$ **end-to-end training**



Fast RCNN – Loss Functions

Classification Loss

$$\mathcal{L}_{\text{cls}} = -\log p_{y^*}$$

- p_{y^*}: softmax prob. for true class y^*
- Includes background class
- Applied to all sampled ROIs

Multi-task Loss

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda\, \mathbf{1}[y^* \neq 0]\, \mathcal{L}_{\text{reg}}$$

- λ=1 (balanced by normalisation)
- Regression only for **non-background**
- Single SGD pass trains everything jointly

Bounding Box Regression Loss

$$\mathcal{L}_{\text{reg}} = \sum_{i \in \{x,y,w,h\}} \text{smooth}_{L_1}(t_i - v_i)$$

$$\text{smooth}_{L_1}(z) = \begin{cases} 0.5z^2 & |z| < 1 \\ |z| - 0.5 & \text{otherwise} \end{cases}$$

Target Parameterisation

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}$$

$$t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}$$

Offsets relative to anchor proposal a.

RCNN vs Fast RCNN – Comparison

Property	RCNN	Fast RCNN
CNN passes/image	≈2000 (per proposal)	**1 (shared map)**
Training stages	3 (CNN + SVM + reg.)	**1 end-to-end**
Test time (VGG16)	≈47 s/image	**≈0.32 s**
Feature storage	Large (disk cache)	**None**
Classifier	Linear SVM	**Softmax (built-in)**
Bbox refinement	Separate step	**Multi-task loss**
Bottleneck	CNN forward pass	**Selective Search**
COCO mAP_{50}	≈35	≈52

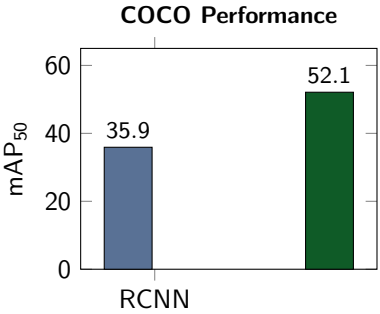
Remaining Bottleneck in Fast RCNN

Selective Search (CPU) takes ≈2 s/image. This motivated **Faster RCNN**, which replaces Selective Search with a learned Region Proposal Network (RPN).

Fast RCNN Results on COCO

Model	**mAP$_{50}$**	**fps**
RCNN (AlexNet)	31.4	0.02
RCNN (VGG16)	35.9	0.02
Fast RCNN (VGG16)	**52.1**	**3.1**
Faster RCNN	*56.8*	*17*

Evaluated on COCO val2014.



Two Paradigms – Side by Side

SSIM-AE on MVTec

- **Task**: Anomaly Detection / Segmentation
- **Dataset**: MVTec AD (industrial)
- **Supervision**: **Unsupervised**
- **Labels**: none at training time
- **Output**: pixel-level anomaly score map
- **Core idea**: reconstruction error signal
- **Loss**: SSIM + ℓ_2 combined
- **Metric**: Pixel AUROC, PRO

RCNN / Fast RCNN on COCO

- **Task**: Object Detection
- **Dataset**: COCO (natural, 80 classes)
- **Supervision**: **Fully supervised**
- **Labels**: bbox + class for every object
- **Output**: bboxes + class + confidence
- **Core idea**: shared CNN + region proposals
- **Loss**: cross-entropy + Smooth-L_1
- **Metric**: mAP_{50}, $\text{mAP}_{50:95}$

Key Takeaways

- **MVTec AD** is the standard benchmark for unsupervised industrial anomaly detection with pixel-level GT masks across 15 categories.
- **SSIM** captures luminance, contrast, *and* structure – critical for detecting fine surface defects where MSE fails.
- **SSIM-AE** trains on normal images only; high reconstruction error signals anomalies with an interpretable pixel-level map.
- **RCNN** proved deep CNNs work for object detection but required 2000 CNN passes and 3 separate training stages.
- **Fast RCNN** fixed those inefficiencies: one shared CNN pass, ROI Pooling, and joint multi-task loss – 147× **faster**.

The two paradigms are **complementary**: anomaly detection needs no defect labels; object detection needs full supervision.

References I

- P. Bergmann et al. *MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection*. CVPR, 2019.
- P. Bergmann et al. *Improving Unsupervised Defect Segmentation by Applying Structural Similarity to Autoencoders*. VISAPP, 2019.
- Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli. *Image Quality Assessment: From Error Visibility to Structural Similarity*. IEEE TIP, 2004.
- R. Girshick, J. Donahue, T. Darrell, J. Malik. *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*. CVPR, 2014.
- R. Girshick. *Fast R-CNN*. ICCV, 2015.
- T.-Y. Lin, M. Maire, S. Belongie, et al. *Microsoft COCO: Common Objects in Context*. ECCV, 2014.
- J. R. R. Uijlings et al. *Selective Search for Object Recognition*. IJCV, 2013.

Thank You!

Questions welcome

your.email@institution.edu